

KIDNEY STONE DETECTION CNN

Agent-facing implementation specification

Purpose - This document is the build contract for the repository. It defines the dataset boundary, canonical CNN architecture, minimal module structure, training procedure, output artifacts, and acceptance criteria. The coding agent should implement this specification directly rather than inventing alternate architectures or infrastructure.

Decision	Locked V1 choice
Task	2D binary classification: Non-Stone (0) vs Stone (1)
Dataset	Mendeley V2 axial CT JPG dataset; 201 patients
Input	384×384×1 grayscale
Model	One canonical custom CNN: 64 → 128 → 256 → 512 channels, 2 convs/block
Head	GAP → Dense192 ReLU → Dropout0.35 → Dense2 Softmax
Training	AdamW, LR 3e-4, weight decay 1e-4, batch 16, max 60 epochs
Validation	Fixed 5-fold patient-wise CV inside 161 development patients
Final test	40 patients, original images only, opened once after lock
Primary metric	OOF ROC-AUC; report sensitivity, specificity, precision, F1, PR-AUC, confusion matrix

Revision intent: replaces the previous 17-page plan with one canonical implementation path. It removes the CNN-S/CNN-M fork, removes conflicting Sigmoid/BCE instructions, reduces configuration and source-file count, and moves 512px / supplied-augmentation experiments out of the required V1 build.

Source dataset reference: Abdalla et al., Data in Brief 59 (2025), 111446; Mendeley Data V2 DOI 10.17632/fwhytt5mzd.2.

1. Scope and non-negotiable decisions

The repository should be small enough to understand in one sitting, but structured so that data logic, model logic, training, and evaluation can change independently. V1 is an image-classification research pipeline, not an MLOps platform and not a deployment product.

Critical data interpretation - Treat the resource as 201 patients with related source-image families, not as 35,457 independent observations. The previous plan reports 3,364 original images (1,577 Stone; 1,787 Non-Stone) and a 35,457-image supplied derivative set.

V1 MUST implement

- Patient-wise outer split: exactly 161 development patients and 40 final-test patients, frozen once.
- Five fixed patient-wise CV folds inside the 161-patient development pool.
- Original images only for every validation fold and for the final test set.
- One canonical 384×384 grayscale CNN with Dense(2) + Softmax output.
- Mild online augmentation only on training images.
- OOF prediction aggregation, OOF-derived decision threshold, explicit candidate lock, then one final-test evaluation.
- Filesystem-based run artifacts with enough provenance to reproduce a result.

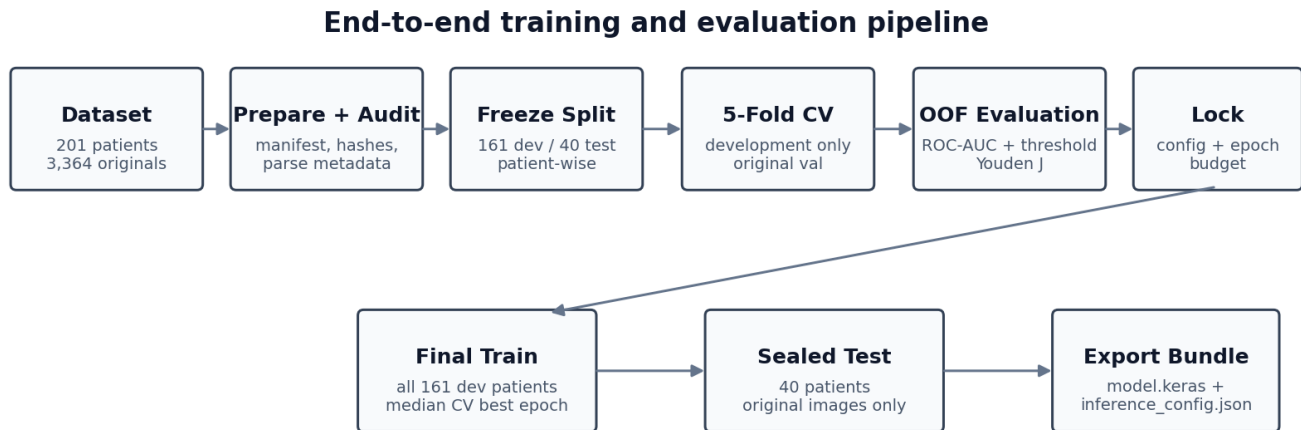
V1 MUST NOT implement

- Random image-level train/test splitting.
- Supplied augmented derivatives in validation or final test.
- A second model family, transfer-learning branch, segmentation model, or broad hyperparameter sweep.
- MLflow, Weights & Biases, a database, distributed scheduler, Docker/Kubernetes, or multi-GPU orchestration.
- DICOM or Hounsfield-unit windowing logic: the supplied archive is JPG, so do not invent unavailable signal.
- Any threshold, epoch, architecture, augmentation, or optimizer choice based on the final 40-patient test set.

Key changes from the previous draft

Previous draft	Updated V1
CNN-S + CNN-M candidates	One canonical model only: kidney_cnn_v1 (~4.79M params).
Dense1 + Sigmoid + BCE in architecture section	Dense2 + Softmax + SparseCategoricalCrossentropy throughout.
baseline.yaml + lab.yaml + final.yaml	One configs/default.yaml; finalization copies an immutable locked config.
Nested data package + seven source modules	Six focused source modules; one responsibility per file.
512px and supplied 35k augmentation in core experiment sequence	Deferred until V1 is complete; not required for first implementation.
Training-time metric complexity	Checkpoint and early-stop on val_loss; compute ROC-AUC and threshold offline from best-checkpoint probabilities.

2. End-to-end pipeline



Hard boundary: the final 40-patient test set is never used for architecture, augmentation, optimizer, epoch, or threshold decisions.

1. Prepare the dataset: parse filenames, build a manifest, verify decodability/dimensions, hash files, and identify patient_id + source_image_id + label + augmentation status.
2. Freeze the outer patient split: 161 development patients / 40 sealed test patients. Persist IDs; never regenerate silently.
3. Freeze five patient-wise folds inside development. Every original/derivative from the same patient inherits the same outer split and fold.
4. Run a smoke test on a small subset to prove decoding, tensor shapes, model forward pass, checkpointing, and artifact writing.
5. Run 5-fold CV at 384×384. Train on original development images plus mild online augmentation; validate on original images only.
6. Reload each fold best checkpoint and save class-1 Stone probabilities for its held-out fold. Concatenate all held-out predictions into one OOF file.
7. Compute OOF metrics and choose the operating threshold from OOF probabilities only. Default threshold rule: maximize Youden J.
8. Explicitly lock the resolved config, threshold, and final epoch budget. Final epoch budget = rounded median of the five best CV epochs.
9. Retrain from scratch on all 161 development patients for the locked epoch budget; do not use the final test for early stopping.
10. Evaluate once on the sealed 40-patient original-only test set, save raw probabilities/metrics/plots, and export the final model bundle.

Hard rule - The final test is a measurement, not a development resource. If the final test looks poor, report it. Do not reopen tuning and then present the same test as unbiased.

3. Repository structure

```

kidney-stone-cnn/
├── README.md
├── requirements.txt
├── .gitignore
├── train.py
├── configs/
│   └── default.yaml
├── src/
│   ├── __init__.py
│   ├── data.py
│   ├── splits.py
│   ├── model.py
│   ├── trainer.py
│   ├── evaluate.py
│   └── utils.py
├── splits/
│   ├── outer_split.csv
│   └── cv_folds.csv
├── tests/
│   ├── test_splits.py
│   └── test_smoke.py
├── data/          # gitignored: local dataset + generated manifest
└── runs/          # gitignored: experiment artifacts/checkpoints

```

This is intentionally the whole V1 repository. Do not add packages merely to mirror textbook layers. Add a new file only when a concern has a separate lifecycle or is likely to change independently.

3.1 File-by-file implementation contract

File	Required responsibilities
train.py	Single CLI/orchestrator. Subcommands: prepare, smoke, cv, finalize. No model internals or metric formulas.
configs/default.yaml	All tunable V1 settings. CLI may override data path, fold number, and batch size only.
src/data.py	Filename parsing, manifest creation, JPG decoding, <code>resize_with_pad</code> , <code>[0,1]</code> scaling, training-only augmentation, <code>tf.data</code> construction.
src/splits.py	Create/load frozen 161/40 patient split, five development folds, split hashes, and leakage assertions.
src/model.py	Define <code>build_model(config)</code> for the canonical CNN only. No I/O, fitting, evaluation, or dataset access.
src/trainer.py	Compile model, create callbacks, train/resume one fold, final retraining, save history/best epoch/checkpoint.
src/evaluate.py	Predict probabilities, OOF merge, threshold selection, metrics, ROC/PR/confusion-matrix plots, final bundle export.
src/utils.py	YAML loading, seed setup, hardware/version metadata, JSON/YAML writes, run IDs, file hashing.
tests/test_splits.py	Synthetic manifest tests proving zero patient/source/hash overlap and stable frozen assignments.
tests/test_smoke.py	Model forward pass + output shape/probability sum; minimal training/checkpoint round-trip on synthetic tensors.

4. Data preparation and split protocol

4.1 Canonical manifest

The prepare command should scan the local dataset once and write `data/manifest.csv`. Every training decision is derived from this manifest rather than from ad-hoc directory traversal.

Column	Meaning
path	Path relative to configured data_dir.
sha256	Exact file content hash.
patient_id	Parsed patient/group identifier.
source_image_id	Canonical original-image family ID, excluding augmentation suffix.
label	0 = Non-Stone; 1 = Stone.
is_original	True only for untouched source image.
augmentation_type	none for original; parsed supplied transform name otherwise.
hospital / metadata	Retain parsed site/metadata fields when available for auditing, not as model input.
height,width,channels	Decoded image properties.

4.2 Integrity checks before any fit()

- 100% parser coverage for files that may enter training/evaluation.
- Every file decodes as expected; corrupt/unreadable files block the run rather than being silently skipped.
- Labels are exactly {0,1}; originals and supplied derivatives are distinguishable.
- No patient appears in more than one outer partition.
- No source_image_id crosses a held-out boundary.
- No exact SHA-256 duplicate crosses a held-out boundary.
- Outer split contains exactly 161 development and 40 test patient IDs.

4.3 Split generation

Generate the patient assignments exactly once. Use group-aware splitting, then audit the resulting image-level class proportions and site composition. If the first deterministic split is pathologically imbalanced, fix the split-generation logic once before training and freeze the accepted CSVs. After the CSVs are committed, training only loads them.

```
splits/outer_split.csv
patient_id,split
P001,development
P002,test
...

splits/cv_folds.csv
patient_id,fold
P001,0
P003,4
... # development patients only
```

No silent regeneration - If a split file exists, the code must load and validate it. It must not generate a new split because a seed changed, a machine changed, or a user reran the command.

5. Input pipeline and augmentation

5.1 Canonical preprocessing

1. Read JPG bytes with TensorFlow and decode deterministically as one grayscale channel.
2. Convert to float32.
3. Resize while preserving aspect ratio and pad to 384×384 using `tf.image.resize_with_pad(..., antialias=True)`.
4. Scale pixel values to [0,1].
5. Return tensor shape (384,384,1) and integer label 0/1.

The exact same decode/resize/scale function is used by training, validation, final evaluation, and exported inference. Augmentation is a separate training-only function layered after canonical preprocessing.

5.2 Mild online augmentation - V1 defaults

Transform	V1 limit	Reason
Rotation	$\pm 10^\circ$	Small acquisition/orientation variation without teaching extreme anatomy invariance.
Translation	$\pm 3\%$ height/width	Small positioning robustness.
Zoom	$\pm 5\%$	Small scale variation.
Contrast	$\pm 10\%$	Modest intensity robustness for JPG data.
Flips	Disabled	Do not assume left-right or superior-inferior symmetry in V1.
Elastic / perspective / piecewise affine	Disabled	Too aggressive for the baseline; only later controlled experiments may justify them.

Default corpus for V1 training: original images belonging to the current training patients. The supplied 35,457 derivative set is indexed but excluded. This avoids conflating more files with more independent patients.

5.3 tf.data order

```

TRAIN: manifest filter → shuffle(seed) → decode/preprocess → augment → batch → prefetch(AUTOTUNE)
VAL:   manifest filter → decode/preprocess → batch → prefetch(AUTOTUNE)
TEST:  manifest filter → decode/preprocess → batch → prefetch(AUTOTUNE)

```

Use parallel map with AUTOTUNE. Dataset cache() is optional and should be enabled only if the actual lab machine benefits; caching must not change dataset semantics.

6. Canonical model architecture

Kidney CNN V1 - canonical architecture



3x3 convolutions, stride 1, same padding, use_bias=False, He normal | ~4.789M total params; ~4.785M trainable

There is one architecture in V1. In code, name it `kidney_cnn_v1`. Every Conv2D uses a 3x3 kernel, stride 1, same padding, use_bias=False, and He-normal initialization. Each convolution is followed by BatchNormalization and ReLU. Each block ends with 2x2 max pooling.

Stage	Operation	Output @ 384
Input	Grayscale canonical preprocessing	384x384x1
Block 1	[Conv64 → BN → ReLU] x2 → MaxPool2D(2)	192x192x64
Block 2	[Conv128 → BN → ReLU] x2 → MaxPool2D(2)	96x96x128
Block 3	[Conv256 → BN → ReLU] x2 → MaxPool2D(2)	48x48x256
Block 4	[Conv512 → BN → ReLU] x2 → MaxPool2D(2)	24x24x512

Stage	Operation	Output @ 384
Head	GlobalAveragePooling2D	512
Dense	Dense(192, He normal) → ReLU → Dropout(0.35)	192
Output	Dense(2, activation="softmax", dtype="float32")	2 probabilities

Output contract - Index 0 = Non-Stone; index 1 = Stone. Save `y_pred[:, 1]` as `stone_probability`. The two Softmax outputs sum to 1. Do not implement a Sigmoid head in V1.

Expected parameter count with convolution biases disabled: approximately 4,788,866 total parameters, of which approximately 4,785,026 are trainable. Global average pooling is mandatory; do not replace it with Flatten, which would create a very large dense head.

7. Training configuration and loop

Hyperparameter	Locked default	Implementation note
Optimizer	AdamW	<code>tf.keras.optimizers.AdamW</code>
Initial learning rate	3e-4	0.0003
Weight decay	1e-4	Do not also add kernel L2 by default.
Loss	<code>SparseCategoricalCrossentropy</code>	Labels are integer 0/1; output is Dense2 Softmax.
Batch size	16 @ 384px	May be reduced for VRAM; changing batch size is the only hardware-specific training override.
Epoch cap	60	Maximum, not target.
Early stopping	<code>val_loss</code> , patience 8	<code>restore_best_weights=False</code> ; reload <code>ModelCheckpoint</code> <code>best.keras</code> explicitly.
LR reduction	<code>val_loss</code> , patience 3, factor 0.5	<code>min_lr = 1e-6</code> .
Checkpoint	best <code>val_loss</code>	Save full .keras model for each fold.
Seed	42	Set Python, NumPy, TensorFlow and shuffle seeds.
Mixed precision	auto	Enable on compatible GPU; final Softmax layer remains float32.

7.1 Fold training sequence

1. Build train and validation manifests from frozen patient assignments.
2. Create fresh model weights for the fold. Never warm-start one fold from another.
3. Compile with AdamW + `SparseCategoricalCrossentropy` + `SparseCategoricalAccuracy`. Keep compile-time metrics minimal.
4. Fit with `ModelCheckpoint(best.keras)`, `EarlyStopping(val_loss, patience=8)`, and `ReduceLROnPlateau(val_loss, patience=3, factor=0.5, min_lr=1e-6)`.
5. After training, reload `best.keras` regardless of the last epoch.
6. Predict all original validation images and save path, `patient_id`, `source_image_id`, `y_true`, `stone_probability`, `predicted_class_at_0_5`.
7. Save `history.csv` and `best_epoch` in fold metadata.

7.2 Why validation loss is the callback monitor

This keeps the training code simple and avoids a custom stateful AUC metric for two-output Softmax. ROC-AUC remains the development selection metric, but it is computed reliably offline from saved probabilities produced by each best checkpoint. Callback monitoring and research evaluation are deliberately separate concerns.

8. OOF evaluation, threshold, and finalization

8.1 OOF aggregation

After all five folds complete, concatenate the five validation-prediction files. Every development image now has exactly one prediction from a model that did not train on its patient. This is the OOF dataset used for model assessment and threshold selection.

Metric	Definition / use
ROC-AUC	Primary threshold-independent ranking metric.
PR-AUC	Complementary ranking metric, useful if prevalence changes.
Sensitivity / Recall	$TP / (TP + FN)$.
Specificity	$TN / (TN + FP)$.
Precision	$TP / (TP + FP)$.
F1	Harmonic mean of precision and recall.
Balanced accuracy	$(Sensitivity + Specificity) / 2$.
Confusion matrix	TN, FP, FN, TP at the locked operating threshold.

8.2 Threshold rule

Use Stone probability `y_pred[:,1]`. Calculate the ROC curve from the pooled OOF predictions and select the threshold that maximizes:

$$\text{Youden J} = \text{sensitivity} + \text{specificity} - 1$$

Save the selected threshold and the OOF metrics that produced it. A 0.5 threshold may be reported as a reference but is not the locked operating threshold unless it happens to be selected.

8.3 Final epoch budget and final train

- Collect the best epoch from each CV fold.
- Compute `final_epochs = round(median(best_epochs))`.
- Write `locked_config.yaml` containing the resolved configuration, threshold, and `final_epochs`.
- Create a new model from scratch and train on all original development images (all 161 patients) using the same online augmentation for exactly `final_epochs`.
- No final-test data enters `fit()`, callbacks, threshold selection, or epoch selection.

8.4 Sealed final evaluation

Run `predict()` once on original images from the 40 final-test patients. Compute all metrics from the saved raw probabilities using the already locked threshold. Save `test_predictions.csv` before any aggregate metric so all reported results can be regenerated later.

9. Configuration and CLI contract

9.1 configs/default.yaml

```
seed: 42

data:
  image_size: 384
  channels: 1
  use_supplied_derivatives: false
  augmentation:
    rotation_deg: 10
    translation_fraction: 0.03
    zoom_fraction: 0.05
    contrast_fraction: 0.10

model:
  name: kidney_cnn_v1
  filters: [64, 128, 256, 512]
  convs_per_block: 2
  kernel_size: 3
  dense_units: 192
  dropout: 0.35

training:
  batch_size: 16
  epochs: 60
  learning_rate: 0.0003
  weight_decay: 0.0001
  early_stopping_patience: 8
  reduce_lr_patience: 3
  reduce_lr_factor: 0.5
  min_lr: 0.000001
  mixed_precision: auto

evaluation:
  threshold_strategy: youden_j_oof
  class_names: [Non-Stone, Stone]
```

9.2 CLI

```
# 1. Create manifest + audit + frozen splits (first machine only)
python train.py prepare --config configs/default.yaml --data-dir D:/kidney_ct

# 2. Verify the entire path quickly
python train.py smoke --config configs/default.yaml --data-dir D:/kidney_ct

# 3. Run all five folds sequentially
python train.py cv --config configs/default.yaml --data-dir D:/kidney_ct

# Or run one frozen fold on a separate workstation
python train.py cv --config configs/default.yaml --data-dir D:/kidney_ct --fold 2

# 4. After reviewing CV outputs, explicitly finalize the selected run
python train.py finalize --run-dir runs/<run_id> --data-dir D:/kidney_ct
```

prepare is the only command allowed to create split CSVs. cv and finalize must fail if required split files are missing or their hashes differ from the run metadata.

10. Run artifacts and reproducibility

```

runs/<run_id>/
├── config_resolved.yaml
├── metadata.json
├── data_manifest_hash.txt
├── split_hashes.json
├── fold_0/
│   ├── best.keras
│   ├── history.csv
│   ├── val_predictions.csv
│   └── fold_metrics.json
├── fold_1/ ... fold_4/
├── oof_predictions.csv
├── oof_metrics.json
├── threshold.json
├── plots/
│   ├── roc_curve.png
│   ├── pr_curve.png
│   ├── confusion_matrix.png
│   └── learning_curves.png
├── locked_config.yaml      # created by finalize
└── final/
    ├── model.keras
    ├── test_predictions.csv
    ├── test_metrics.json
    ├── plots/
    └── bundle/
        ├── model.keras
        ├── inference_config.json
        └── provenance.json

```

metadata.json should stay compact: run ID, command, timestamp, Git commit + dirty state, Python/TensorFlow versions, GPU name, seed, dataset manifest hash, outer split hash, CV fold hash, and elapsed time. Completed run directories are immutable; reruns receive a new run ID.

10.1 Inference bundle contract

inference_config.json must be sufficient for a later prediction script to reproduce model input and class interpretation without reading training code.

```

{
  "image_size": 384,
  "channels": 1,
  "resize": "resize_with_pad",
  "scale": "0_1",
  "class_names": ["Non-Stone", "Stone"],
  "stone_class_index": 1,
  "threshold": <locked OOF threshold>
}

```

11. Agent implementation sequence

Phase	Build	Exit criterion
P0 - Bootstrap	Repo tree, requirements, YAML loader, CLI skeleton.	train.py --help works in a clean venv.
P1 - Data	Filename parser, manifest, hashes, decode audit.	Real dataset produces a complete manifest with zero unexplained parse/decode failures.
P2 - Splits	161/40 outer split, five dev folds, leakage assertions.	CSV files frozen; tests prove zero patient/source/hash overlap.
P3 - Pipeline	Canonical preprocessing + mild augmentation + tf.data.	One batch has expected shape/range and validation has no augmentation.
P4 - Model	kidney_cnn_v1 exactly as section 6.	Model summary matches output shapes and ~4.79M params.

Phase	Build	Exit criterion
P5 - Smoke	Tiny real subset, 1 fold, 1-2 epochs.	Checkpoint reload + probability export succeeds.
P6 - CV	All five folds, callbacks, histories, predictions.	Five best checkpoints + five validation prediction files exist.
P7 - OOF	Merge predictions, metrics, threshold, plots.	OOF metrics and threshold.json reproducible from CSV.
P8 - Lock/final	Median epoch budget, all-dev retrain, sealed test.	Final test metrics created once from saved probabilities.
P9 - Release	README quick start + final bundle.	Fresh clone can run prepare/smoke/CV with a supplied data path.

11.1 Minimal tests

- Split integrity: synthetic groups prove no patient/source/hash leakage.
- Model contract: random input (2,384,384,1) returns shape (2,2); every row is finite, in [0,1], and sums to ~1.
- Metric correctness: toy y_{true} /probabilities produce known confusion-matrix counts and specificity/sensitivity.
- Checkpoint round-trip: a tiny synthetic fit saves best.keras; reloaded model produces valid probabilities.

Do not build a large testing framework. These tests protect the high-risk boundaries: leakage, output semantics, metrics, and serialization.

12. Definition of done

Repository is complete only when - All acceptance criteria below pass on the real dataset and a clean clone.

- The dataset manifest is reproducible and every used file is decodable and traceable to patient/source IDs.
- The committed outer split contains 161 development and 40 test patients with zero patient/source/hash overlap.
- The five development folds are frozen and reused by every run.
- Validation/test contain original images only; V1 training does not use the supplied 35k derivatives.
- The model is exactly Dense2 Softmax with integer labels and SparseCategoricalCrossentropy.
- All five CV folds train independently and reload their best val_loss checkpoints.
- OOF predictions cover the full development pool exactly once and are the sole basis for the locked threshold.
- Final epoch budget is derived from CV; the test set does not influence training duration.
- The final 40-patient test is evaluated once after an explicit finalize/lock action.
- Raw test probabilities, threshold, metrics, confusion matrix, ROC/PR curves, config, split hashes, dataset hash, Git state, and model bundle are saved.
- README documents only the commands required to prepare, smoke-test, run CV, and finalize.

12.1 Deferred experiments - not part of V1 implementation

Only after the canonical pipeline is complete and leakage-free should the project consider these experiments. They must reuse the exact frozen split/folds and change one variable at a time.

- 512×512 input resolution if both training and validation indicate underfitting or loss of small-detail signal.
- Supplied 35,457 derivative images as training-only data, while keeping validation/test original-only.
- Regularization changes (dropout / weight decay) only if a clear train-vs-CV generalization gap exists.
- A pretrained benchmark only as a comparison, not as a replacement for the custom V1 baseline.

References

- [1] P. A. Abdalla et al., "Kidney stone detection via axial CT imaging: A dataset for AI and deep learning applications," Data in Brief, vol. 59, 111446, 2025. DOI: 10.1016/j.dib.2025.111446
- [2] Mendeley Data, "Axial CT Imaging Dataset for AI-Powered Kidney Stone Detection: A Resource for Deep Learning Research," Version 2. DOI: 10.17632/fwhytt5mzd.2
- [3] TensorFlow/Keras documentation: tf.data, mixed precision, ModelCheckpoint, EarlyStopping, ReduceLROnPlateau, AdamW.
- [4] scikit-learn documentation: group-aware splitting, ROC/PR metrics and confusion-matrix metrics.